Rapport de Projet — Bunny Plush Designer

Générateur Paramétrique de Patron de Peluche Lapin en 3D

Auteurs :

- Hocine BOUROUIH
- Mohamed AIDAOUI
- Rayan BELDI

Date : Mars 2026

Table des matières

1. Introduction
2. Objectif du projet
3. Choix technologiques et justifications
4. Architecture du projet
5. Solution choisie : le patronage paramétrique
6. Explication de l'algorithme
7. Fonctionnalités réalisées
8. Exports et couture
9. Diagramme d'architecture
10. Conclusion

---

1. Introduction

Ce projet est une application web qui permet de concevoir une peluche de lapin en 3D, de personnaliser ses dimensions et son apparence en temps réel, puis d'exporter des patrons de couture prêts à imprimer au format SVG et PDF. L'ensemble fonctionne intégralement dans le navigateur, sans serveur, sans base de données, et sans étape de compilation (*zero build step*).

---

2. Objectif du projet

Réaliser un **outil de conception paramétrique 3D** permettant à un utilisateur de :

1. Visualiser en temps réel un lapin en peluche dans un viewport 3D interactif.
2. Modifier les dimensions (taille du corps, longueur des oreilles, taille de la queue, rondeur) et le style (couleur, style d'oreilles, accessoire) via un panneau de contrôles.

3. Exporter automatiquement les **patrons de couture 2D** correspondant aux dimensions choisies, avec toutes les annotations professionnelles nécessaires à la couture (repères de correspondance, sens du poil, marges de couture, zones à laisser ouvertes, guide d'assemblage).

---

3. Choix technologiques et justifications

Technologie	Rôle	Justification
HTML/CSS/JS natif	Structure, style, logique	Aucune dépendance framework (React, Vue…) → projet léger, portable, pas de build
Three.js v0.160.1	Rendu 3D WebGL	Bibliothèque de référence pour le rendu 3D dans le navigateur. Chargé via CDN + ES modules `importmap`
jsPDF 2.5.1	Génération PDF côté client	Permet de générer des PDF multipages tuilés directement dans le navigateur, sans serveur
ES Modules	Architecture modulaire	`import/export` natifs du navigateur — pas besoin de bundler (Webpack, Vite…)
CSS custom properties	Thème dark cohérent	Variables CSS centralisées dans `:root` pour maintenir la cohérence visuelle
Google Fonts (Inter)	Typographie	Police moderne et lisible pour l'interface utilisateur

Pourquoi pas Blender ?

Nous avons choisi de **développer notre propre générateur de patrons (in-house)**, en utilisant la technique de **patronage paramétrique** (*parametric pattern drafting*) : les pièces sont calculées algorithmiquement en JavaScript à partir des paramètres saisis par l'utilisateur (taille du corps, longueur des oreilles, rondeur, taille de la queue), **sans passer par Blender**.

Raisons de ce choix :

- **Temps réel** : les patrons se recalculent instantanément à chaque modification d'un slider — impossible avec un workflow Blender export/import.
- **Portabilité** : l'application tourne dans n'importe quel navigateur, sans installer Blender ni aucun logiciel tiers.
- **Maîtrise totale** : nous contrôlons le format de sortie (SVG vectoriel, PDF tuilé A4), les annotations de couture, les repères de correspondance — éléments difficiles à extraire automatiquement d'un maillage 3D.
- **Légèreté** : pas de fichiers `.blend` volumineux, pas de pipeline de conversion → le projet entier tient en ~1 400 lignes de code.

---

4. Architecture du projet

Structure des fichiers

```
parametric-garments/
├── index.html              ← Point d'entrée HTML
```

```
├── style.css              ← Thème dark, layout flexbox, contrôles
├── README.md              ← Documentation utilisateur
├── RAPPORT.md             ← Ce rapport
└── js/
    ├── parameters.js      ← Définitions des paramètres + EventTarget store
    ├── preview3d.js       ← Scène Three.js + construction du lapin 3D
    ├── patternGenerator.js ← Algorithme de patronage paramétrique
    ├── exporter.js        ← Export SVG + PDF avec annotations couture
    └── main.js            ← Point d'entrée JS : câblage UI ↔ 3D ↔ export
```

Rôle de chaque fichier

Fichier	Lignes	Rôle
`parameters.js`	~120	**Source unique de vérité.** Définit les 7 paramètres (bodySize, earLength, tailSize, chubbiness, earStyle, color, accessory), leurs bornes min/max, leurs valeurs par défaut. Fournit un `ParameterStore` basé sur `EventTarget` pour notifier les changements aux consommateurs (preview 3D, pattern generator). Exporte la palette `BUNNY_COLORS` (6 couleurs hex).
`preview3d.js`	~523	**Moteur 3D.** Initialise la scène WebGL (renderer, caméra, lumières 3 points, sol, grille). Génère des textures peluche procédurales par canvas (bruit doux imitant la fourrure). Construit le lapin à partir de primitives simples (SphereGeometry, CylinderGeometry, TorusGeometry) : corps, ventre, tête, joues, museau, nez, yeux (avec reflet), oreilles (3 styles), bras (avec coussinets), jambes (avec semelles), queue pompom, moustaches (6 fils). Gère les 3 accessoires (nœud papillon, écharpe, ruban).
`patternGenerator.js`	~325	**Cœur algorithmique.** Calcule les 6 types de pièces de patron 2D (corps, tête, oreille, bras, jambe, queue) en fonction des paramètres. Chaque pièce contient : contour de coupe, contour avec marge de couture (1.5 cm), ligne de droit fil, flèche de sens du poil, repères lettrés (A–E) de correspondance, zones d'ouverture, note d'assemblage. Exporte les 12 étapes d'assemblage en français.
`exporter.js`	~485	**Export SVG + PDF.** Convertit les pièces en SVG vectoriel (à l'échelle 1:1, 96 dpi) avec légende, titre, barre 10 cm, repères lettrés, flèches sens du poil, marques d'ouverture. Génère un PDF A4 paysage tuilé avec chevauchement 8 mm, repères d'assemblage aux coins, et une page portrait finale de guide d'assemblage (liste de coupe, 12 étapes, légende des symboles, conseils de couture).
`main.js`	~168	**Chef d'orchestre.** Génère dynamiquement les contrôles UI (sliders, selects) à partir de `PARAM_DEFS`. Câble les événements `change` du store aux mises à jour 3D debounced (30 ms). Gère les boutons export SVG/PDF. Observe le resize du viewport via `ResizeObserver`.
`index.html`	~98	**Squelette HTML.** Layout 2 colonnes (panneau 320 px + viewport 3D). Charge Three.js via `importmap`, jsPDF via CDN UMD.
`style.css`	~381	**Thème visuel.** Design dark, 28 variables CSS, layout flexbox, sliders custom stylisés, boutons avec icônes, viewport avec overlay (badge « 3D Preview », hint « Drag to orbit »).

5. Solution choisie : le patronage paramétrique

Qu'est-ce que le patronage paramétrique ?

Le **patronage paramétrique** (*parametric pattern drafting*) est une technique où les pièces d'un patron de couture ne sont pas dessinées à la main, mais **calculées mathématiquement** à partir de paramètres numériques (dimensions, proportions). C'est la méthode utilisée par l'industrie textile moderne pour la gradation automatique des tailles.

Notre approche

Dans notre projet, nous appliquons cette technique au domaine des peluches : chaque pièce du lapin (corps, tête, oreille, bras, jambe, queue) est définie comme une **forme géométrique paramétrée** dont les dimensions dépendent directement des paramètres utilisateur.

Paramètres d'entrée → Formes géométriques paramétrées :

Paramètre	Effet sur le patron
`bodySize` (8–30 cm)	Contrôle le facteur d'échelle global `S = bodySize / 16` qui multiplie toutes les dimensions
`chubbiness` (50–150 %)	Multiplie la **largeur** des pièces (corps, bras, jambes) sans toucher à la hauteur → lapin plus fin ou plus dodu
`earLength` (4–18 cm)	Définit directement la hauteur de la pièce oreille
`tailSize` (1–6 cm)	Définit le rayon de la pièce queue

Le pipeline complet :

```
Paramètres utilisateur (sliders)
        │
        ▼
  ParameterStore (EventTarget)
        │
        ├──────────────────────────────┐
        ▼                              ▼
  Preview 3D (Three.js)          Pattern Generator (2D)
  ↳ _buildBunny(params)           ↳ generatePattern(params)
  ↳ Primitives 3D (sphères,      ↳ Formes 2D (ellipses,
    cylindres, torus)               rectangles arrondis)
  ↳ Rendu temps réel WebGL       ↳ Pièces avec marges,
                                    repères, annotations
        │                               │
        ▼                               ▼
  Canvas 3D interactif            Exporter SVG / PDF
  (orbit, zoom, pan)              (téléchargement)
```

---

6. Explication de l'algorithme

6.1 Génération des contours de pièces

Chaque pièce est calculée par une fonction dédiée dans `patternGenerator.js`. L'algorithme suit les étapes suivantes :

Étape 1 : Calcul du facteur d'échelle

```
const S    = p.bodySize / 16;   // facteur d'échelle (1.0 pour bodySize=16)
const chub = p.chubbiness / 100; // facteur de rondeur (1.0 pour 100%)
```

La valeur par défaut de `bodySize` est 16 cm, donc `S = 1.0` est la taille de référence. Toutes les dimensions sont multipliées par `S`.

Étape 2 : Génération du contour (ellipse ou rectangle arrondi)

Pour les pièces à contour elliptique (corps, tête, bras, jambe, queue), nous échantillonnons une ellipse paramétrée :

```
function ellipsePoints(rx, ry, n) {
  const pts = [];
  for (let i = 0; i < n; i++) {
    const angle = (i / n) * Math.PI * 2;
    pts.push([rx * Math.cos(angle), ry * Math.sin(angle)]);
  }
  return pts;
}
```

- `rx`, `ry` = demi-rayons de l'ellipse (dépendent de `S` et `chub`)
- `n` = nombre de points d'échantillonnage (16–24 pour une courbe lisse)

Pour les oreilles (forme allongée), nous utilisons un rectangle arrondi (`roundedRect`) qui génère 4 coins à rayon paramétré.

Étape 3 : Offset du polygone (marge de couture)

La marge de couture (1.5 cm) est calculée par **offset par bissectrice des normales d'arêtes** :

```
function expandOutline(pts, amount) {
  // 1. Déterminer le sens de rotation (aire signée)
  // 2. Pour chaque sommet :
  //    a. Calculer la normale de l'arête entrante (prev → pt)
  //    b. Calculer la normale de l'arête sortante (pt → next)
  //    c. Bissectrice = moyenne des deux normales
  //    d. Longueur miter = amount / cos(demi-angle), clampée à 3×
  //    e. Nouveau point = pt + bissectrice × longueur miter
}
```

Cet algorithme est une implémentation de l'**offset de Minkowski** simplifié pour les polygones convexes. Le clamping à 3× empêche les pointes extrêmes aux angles aigus.

Étape 4 : Enrichissement des métadonnées

Chaque pièce est retournée avec ses annotations professionnelles :

```
return {
  id: 'body',
  label: 'Corps (×2)',           // nom + nombre de découpes
  number: '1/6',                 // numéro de pièce
  outline,                       // contour de coupe (polygone)
  withSeamAllowance,             // contour + marge 1.5 cm
  grainLine: [[x1,y1],[x2,y2]], // flèche droit-fil
  furDirection: [[...],[...]],   // flèche sens du poil
  notches: [                     // repères de correspondance
    { pos: [x,y], letter: 'A', label: 'Cou → Tête' },
    ...
  ],
  openings: [                    // zones à laisser ouvertes
    { from: [x,y], to: [x,y], label: 'Ouvert pour rembourrer' },
  ],
  assemblyNote: '...',           // note d'assemblage
};
```

6.2 Rendu 3D du lapin

Le lapin 3D est construit dans `preview3d.js` par la méthode `_buildBunny(p)` qui assemble des **primitives Three.js simples** :

Partie	Géométrie	Remarque
Corps	`SphereGeometry` (aplatie Y×1.1, X×chub)	Ovale vertical
Ventre	`SphereGeometry` (légèrement plus petit)	Couleur accent
Tête	`SphereGeometry`	Positionnée au sommet du corps
Joues	2 × `SphereGeometry`	Dépassent sur les côtés
Yeux	2 × `SphereGeometry` + reflet blanc	Noir + point brillant
Nez	`SphereGeometry` (rose)	Triangle inversé
Museau	`SphereGeometry` (aplati Z)	Zone blanche du nez
Oreilles	2 × `CylinderGeometry` + accent intérieur	3 styles de pose
Bras	2 × `SphereGeometry` (aplati) + coussinets	Côtés du corps
Jambes	2 × `SphereGeometry` (aplati) + semelles	Bas du corps
Queue	`SphereGeometry`	Pompom arrière
Moustaches	6 × `CylinderGeometry` fins	3 par côté

Textures procédurales : les textures de fourrure sont générées par canvas HTML5 (6000 points aléatoires + lignes directionnelles → aspect duveteux), puis appliquées comme `CanvasTexture` avec bump mapping (`bumpScale: 0.015`) et rugosité élevée (`roughness: 0.95`).

Éclairage : système 3 points (key light avec shadows, fill light bleutée, rim light arrière) + hémisphère + ambiante.

6.3 Export SVG et PDF

SVG

Le SVG est généré ligne par ligne dans `buildSVGString()` :

1. **Conversion de coordonnées** : patron (cm, Y-haut) → SVG (px, Y-bas) via `CM_TO_PX = 37.795` (96 dpi) et inversion Y.
2. **Disposition** : pièces côte à côte avec espacement de 4 cm.
3. **Annotations rendues** : contour dashed (marge), contour plein (coupe), flèches grain/sens du poil (SVG markers), cercles rouges lettrés (repères), lignes orange (ouvertures), légende encadrée.

PDF

Le PDF est généré par jsPDF avec tuilage A4 :

1. **Tuilage** : si une pièce dépasse la zone imprimable (273 × 186 mm), elle est découpée en tuiles avec chevauchement de 8 mm.
2. **Repères** : croix + cercle aux 4 coins de chaque page pour aligner les feuilles.
3. **Page d'instructions** : dernière page en portrait A4 contenant la liste de coupe (20 pièces), les 12 étapes d'assemblage, la légende des symboles, et 8 conseils de couture.

---

7. Fonctionnalités réalisées

7.1 Prévisualisation 3D temps réel

- Rendu WebGL avec Three.js (ACES filmic tone mapping, PCFSoftShadowMap)
- Contrôles orbitaux (rotation, zoom, pan) via `OrbitControls`
- Mise à jour debounced à 30 ms (fluidité garantie lors du déplacement des sliders)
- Textures procédurales imitant la fourrure peluche

7.2 Personnalisation paramétrique

Catégorie	Paramètres
Dimensions	Taille du corps (8–30 cm), Longueur des oreilles (4–18 cm), Taille de la queue (1–6 cm), Rondeur (50–150 %)
Style	Style d'oreilles (droites / tombantes / lop), Couleur (6 choix), Accessoire (nœud papillon / écharpe / ruban)

7.3 Patron de couture intelligent

- 6 types de pièces → 20 morceaux à découper
- Marge de couture 1.5 cm calculée par offset polygonal
- **Repères de correspondance lettrés** (A↔A pour cou, B↔B pour bras, C↔C pour jambes, D↔D pour queue, E↔E pour oreilles)
- **Sens du poil** (flèche verte)
- **Zones d'ouverture** (tirets orange)

- **Droit-fil** (flèche grise tiretée)
- Numérotation des pièces (1/6 à 6/6)

7.4 Exports print-ready

- **SVG vectoriel** à l'échelle 1:1 avec légende complète
- **PDF A4 tuilé** avec repères d'assemblage, chevauchement 8 mm
- **Page guide d'assemblage** incluse dans le PDF (12 étapes + conseils)

---

8. Exports et couture

Quel format utiliser ?

Format	Usage	Avantage
PDF	Impression maison (A4)	Tuilé automatiquement, repères d'alignement, guide d'assemblage inclus
SVG	Traceur / découpeuse laser ou Cricut	Une seule feuille à l'échelle, vectoriel pur, éditable dans Inkscape

Annotations de couture

Symbole	Description
Trait plein noir	Ligne de coupe
Tirets gris	Marge de couture (1.5 cm)
Flèche grise (tirets)	Droit-fil (sens de la chaîne du tissu)
Flèche verte pleine	Direction du poil (fourrure)
Cercle rouge + lettre	Repère de correspondance (A↔A, B↔B...)
Tirets orange	Zone à laisser ouverte (rembourrage / retournement)

Liste de coupe

N°	Pièce	Quantité
1/6	Corps	×2 (devant + dos)
2/6	Tête	×2 (devant + dos)
3/6	Oreille	×4 (2 ext. + 2 int. accent)
4/6	Bras	×4 (2 paires G/D)
5/6	Jambe	×4 (2 paires G/D)
6/6	Queue	×2
Total		**20 pièces**

---

9. Diagramme d'architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                              index.html                          │
│   ┌──────────────┐     ┌──────────────────────────────────────┐  │
```

```
│  │  Panneau UI   │    │       Viewport 3D (canvas)        │ │
│  │  (sliders,    │    │                                   │ │
│  │   selects)    │    │    Three.js WebGL                 │ │
│  │               │    │    OrbitControls                  │ │
│  └───────┬───────┘    └──────────────▲────────────────────┘ │
│          │                            │                      │
└──────────┼────────────────────────────┼──────────────────────┘
           │                            │
           ▼                            │
┌─────────────────────┐       ┌──────────┴──────────┐
│   parameters.js     │       │    preview3d.js     │
│                     │       │                     │
│  PARAM_DEFS[]       │──────►│  Preview3D class    │
│  BUNNY_COLORS{}     │       │  _buildBunny(params)│
│  ParameterStore     │       │  _buildBowtie()     │
│  (EventTarget)      │       │  _buildScarf()      │
│                     │       │  _buildRibbon()     │
│  .set() → 'change'  │       │  _makePlushMat()    │
└──────────┬──────────┘       └─────────────────────┘
           │
           │  params.getAll()
           │
           ▼
┌─────────────────────┐       ┌────────────────────┐
│ patternGenerator.js │──────►│     exporter.js    │
│                     │       │                    │
│ generatePattern(p)  │       │ exportSVG(p)       │
│ buildBodyPiece()    │       │ exportPDF(p)       │
│ buildHeadPiece()    │       │ buildSVGString()   │
│ buildEarPiece()     │       │ Guide d'assemblage │
│ buildArmPiece()     │       │ Légende / conseils │
│ buildLegPiece()     │       │                    │
│ buildTailPiece()    │       │ ↓ Téléchargement   │
│ expandOutline()     │       │ bunny-plush-pattern│
│ ellipsePoints()     │       │ .svg / .pdf        │
│ roundedRect()       │       └────────────────────┘
│ ASSEMBLY_STEPS[]    │
└─────────────────────┘
```

Flux de données

```
Utilisateur déplace un slider
       │
       ▼
main.js : slider.addEventListener('input')
       │
       ▼
params.set('bodySize', 22)
       │
       ▼
ParameterStore.dispatchEvent('change')
       │
```

```
       ├──────────────────────────┐
       ▼                          ▼
debouncedUpdate (30ms)      updatePatternInfo()
       │                    ↳ affiche infos texte
       ▼
preview.update(params.getAll())
       │
       ▼
_buildBunny({bodySize:22, ...})
       │
       ▼
Nouveau mesh 3D affiché dans le canvas
```

```
Utilisateur clique "Export PDF"
       │
       ▼
exporter.exportPDF(params.getAll())
       │
       ▼
patternGenerator.generatePattern(params)
       │
       ├── buildBodyPiece(S, chub, 6)    → outline + metadata
       ├── buildHeadPiece(S, chub, 6)    → outline + metadata
       ├── buildEarPiece(S, earLen, 6)   → outline + metadata
       ├── buildArmPiece(S, chub, 6)     → outline + metadata
       ├── buildLegPiece(S, chub, 6)     → outline + metadata
       └── buildTailPiece(S, tailSz, 6)  → outline + metadata
       │
       ▼
jsPDF : pages tuilées A4 paysage
       + repères d'assemblage
       + annotations couture
       + page guide d'assemblage (portrait)
       │
       ▼
Téléchargement : bunny-plush-pattern.pdf
```

---

10. Conclusion

Ce projet démontre qu'il est possible de réaliser un **outil de conception paramétrique complet** — de la visualisation 3D temps réel jusqu'aux patrons de couture imprimables — en utilisant uniquement les technologies web standards (HTML, CSS, JavaScript, WebGL via Three.js), sans recours à des outils lourds comme Blender et sans serveur backend.

Le choix du **patronage paramétrique algorithmique** (in-house, sans Blender) s'est avéré particulièrement adapté car il permet :

- Une **réactivité instantanée** : les patrons se recalculent à chaque mouvement de slider.

- Une **précision des annotations** (repères, sens du poil, ouvertures) impossible à obtenir par simple dépliage UV d'un mesh 3D.
- Une **portabilité totale** : l'application fonctionne hors-ligne dans n'importe quel navigateur moderne.